Python Cheat Sheet - Omar Elgammal

File Handling

```
# Writing to a text file
with open('data.txt', 'w') as f:
    f.write("Hello Python\n")
    f.write("Second Line")
# writing to a CSV
with open('team.csv', mode='w', newline='') as file:
   fieldnames = ['Name', 'Role', 'Salary']
   writer = csv.DictWriter(file, fieldnames=fieldnames)
   # Writing the header row
   writer.writeheader()
   # Writing each dictionary as a row
   for row in data:
       writer.writerow(row)
# Reading from a text file
with open('data.txt', 'r') as f:
    content = f.read()       # Read entire file
    # lines = f.readlines()  # Read as list of lines
    # line = f.readline()    # Read single line

# Reading csv
with open('employees.csv', mode='r') as file:
   reader = csv.reader(file)
   # Skip the first row (header)
   next(reader)
   # Iterate through each row in the CSV
   for row in reader:
       print(f"Employee Name: {row[0]}, Department: {row[1]}")


data = [
    {'Name': 'Alice', 'Role': 'Dev', 'Salary': 90000},
    {'Name': 'Bob', 'Role': 'Designer', 'Salary': 85000}
]

# Writing to a CSV using DictWriter
with open('team.csv', mode='w', newline='') as file:
    fieldnames = ['Name', 'Role', 'Salary']
    writer = csv.DictWriter(file, fieldnames=fieldnames)

    writer.writeheader()           # Writes the keys as the header row
    for row in data:
        writer.writerow(row)       # Writes each dictionary as a row
    # writer.writerows(data)       # Alternative: write all rows at once

# Reading from a CSV using DictReader
with open('team.csv', mode='r') as file:
    reader = csv.DictReader(file)  # Automatically uses the first row as keys
```

```
    for row in reader:
        # Access elements by column name
        print(f"Employee: {row['Name']}, Role: {row['Role']}")
```

String manipulation

```
s = "  Hello Python  "

# Basic Operations
s.strip()                # Remove leading/trailing whitespace
s.lower()                # Convert to lowercase
s.upper()                # Convert to uppercase
s.title()                # Capitalize first letter of each word
s.replace("Python", "AI")# Replace substring

# Splitting and Joining
s.split(" ")             # Split into a list based on delimiter
"-".join(['A', 'B', 'C'])# Join iterable with a separator (e.g., 'A-B-C')

# Formatting & Slicing
name = "Alice"
age = 30
f"Name: {name}, Age: {age}" # f-string interpolation
s[2:7]                   # Slice characters from index 2 to 6
s[::-1]                  # Reverse the string
```

Dictionary Operations

```
# Accessing and Modifying
d.get('salary', 0)       # Safely access value
d['salary'] = 90000      # Add or update a key-value pair

# Dictionary Methods
d.keys()                 # View object of all keys
d.values()               # View object of all values
d.items()                # View object of all (key, value) tuples
d.update({'level': 4})   # Update multiple key-value pairs at once
role = d.pop('role')     # Remove key and return its value
del d['level']           # Delete key-value pair

# Dictionary Comprehension
squares = {x: x**2 for x in range(1, 6)}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Datetime handling

```
import datetime
from datetime import timedelta

# Current Date & Time
```

```
now = datetime.datetime.now()          # Current date and time
today = datetime.date.today()          # Current date only

# Formatting (Datetime to String)
# %Y: Year, %m: Month, %d: Day
# %H: Hour (24hr), %M: Minute, %S: Second
formatted_str = now.strftime("%Y-%m-%d %H:%M:%S")

# Parsing (String to Datetime)
date_str = "2023-10-31"
parsed_date = datetime.datetime.strptime(date_str, "%Y-%m-%d")

# Timedeltas (Date Math)
tomorrow = today + timedelta(days=1)  # Add 1 day
last_week = now - timedelta(weeks=1)  # Subtract 1 week
difference = tomorrow - today         # Returns a timedelta object
```

3. NumPy Cheat Sheet

Creating Arrays

```
import numpy as np
np.array([1, 2, 3])                  # 1D array
np.array([(1, 2, 3), (4, 5, 6)])     # 2D array
np.zeros((3, 4))                     # Array of zeros
np.ones((2, 3, 4), dtype=np.int16)   # Array of ones
np.arange(10, 25, 5)                 # Evenly spaced (start, stop, step)
np.linspace(0, 2, 9)                 # Evenly spaced (number of samples)
np.full((2, 2), 7)                   # Constant array
np.eye(2)                            # 2x2 Identity matrix
np.random.random((2, 2))             # Random values
np.empty((3, 2))                     # Empty array (uninitialized)
np.random.rand(2,3)            # 2x3 array of random floats [0,1)
np.random.randint(0,10,(2,2))  # 2x2 array of random ints [0,10)
# Random Sampling
np.random.seed(0)                      # set seed
np.random.rand(3,3)                    # uniform random [0,1)
np.random.randn(3,3)                   # standard normal random
np.random.randint(0,10,5)              # random integers
np.random.choice([1,2,3,4], size=3)  # random selection
```

Inspecting Arrays

```
a.shape             # Array dimensions
len(a)              # Length of array
b.ndim              # Number of array dimensions
e.size              # Number of array elements
b.dtype             # Data type of array elements
b.dtype.name        # Name of data type
b.astype(int)       # Convert type
```

Reshaping & Stacking

```
a.reshape(4,1)                      # reshape
a.flatten()                          # flatten to 1D
np.ravel(a)                           # flatten (view if possible)
np.vstack([a,b])                      # stack arrays vertically
np.hstack([a,b])                      # stack arrays horizontally
np.concatenate([a,b],axis=0)         # concatenate along axis
np.repeat(a,2,axis=0)                 # repeat rows
np.tile(a,(2,2))                      # tile array
```

Mathematics

```
# Functions
np.exp(b)                       # Exponentiation
np.sqrt(b)                      # Square root
np.sin(a)                       # Sine
np.cos(b)                       # Cosine
np.log(a)                       # Natural log
e.dot(f)                        # Dot product
np.add(a,1)                     # element-wise addition
np.subtract(a,1)                # element-wise subtraction
np.multiply(a,2)                # element-wise multiplication
np.divide(a,2)                  # element-wise division
np.power(a,2)                   # element-wise square
np.sum(a)                        # sum all elements
np.sum(a,axis=0)                 # sum along columns
np.mean(a)                        # mean of all elements
np.mean(a,axis=1)                 # mean along rows
np.max(a)                          # maximum
np.min(a)                          # minimum
np.std(a)                           # standard deviation
np.var(a)                           # variance
```

Comparison

```
a == b              # Element-wise comparison
a < 2               # Element-wise comparison
np.array_equal(a, b) # Check if arrays are equal
```

Aggregate Functions

```
a.sum()             # Array-wise sum
a.min()             # Array-wise min value
b.max(axis=0)       # Max value of an array row
b.cumsum(axis=1)    # Cumulative sum
a.mean()            # Mean
np.median(b)        # Median
np.corrcoef(a)      # Correlation coefficient
np.std(b)           # Standard deviation
```

Subsetting, Slicing, Indexing

```
# Subsetting
a[2]                # Select element at index 2
b[1, 2]             # Select element at row 1, col 2
# Slicing
a[0:2]              # Select items at index 0 and 1
b[0:2, 1]           # Select items at rows 0 and 1 in column 1
b[:1]               # Select all items at row 0
a[::-1]             # Reversed array
# Boolean Indexing
a[a < 2]            # Select elements less than 2
# Fancy Indexing
b[[1, 0, 1, 0], [0, 1, 2, 0]] # Select specific elements
```

Array Manipulation

```
i = np.transpose(b) # Permute dimensions
i.T                 # Permute dimensions
b.ravel()           # Flatten the array
g.reshape(3, 2)     # Reshape, but don't change data
h.resize((2, 6))    # Return new array with shape
np.append(h, g)     # Append items to array
np.insert(a, 1, 5)  # Insert items
np.delete(a, [1])   # Delete items
```

Combining & Splitting

```
# Combining
np.concatenate((a, d), axis=0)  # Concatenate arrays
np.vstack((a, b))               # Stack vertically (row-wise)
np.hstack((e, f))               # Stack horizontally (column-wise)
np.column_stack((a, d))         # Stack column-wise

# Splitting
np.hsplit(a, 3)     # Split horizontally at 3rd index
np.vsplit(c, 2)     # Split vertically at 2nd index
```

Miscellaneous

```
np.unique(a)                        # unique elements
np.sort(a)                          # sort array
np.argsort(a)                       # indices of sorted array
np.isin(a,[1,2])                    # check membership
np.where(a>2)                       # indices where condition is True
np.copy(a)                           # deep copy
np.round(np.pi,3)                    # round number to 3 decimals
np.clip(a,0,2)                       # limit values to [0,2]
np.nan, np.isnan(a)                  # nan and check nan
```

4. Matplotlib Cheat Sheet

Basic Workflow

1. **Prepare Data:** `x = [1,2,3]`, `y = [10,20,30]`
2. **Create Plot:** `fig = plt.figure()`
3. **Plot:** `ax.plot(x, y)`
4. **Customize:** `ax.set_title('Title')`
5. **Save:** `plt.savefig('plot.png')`
6. **Show:** `plt.show()`

Subplots

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(2,1,figsize = (10,8) )
fig.subplots_adjust(wspace=0.5, hspace=0.3) # Adjust spacing
ax[0].plot(hours,counts_arr,'k',label = 'Vehicle Volume')
ax[0].axhline(y=100, color='r', linestyle='--', label='High Traffic')
```

Plotting Routines (1D Data)

```
lines = ax.plot(x, y)              # Connect points with lines
ax.scatter(x, y)                   # Unconnected points
ax.bar(x, y)                       # Vertical rectangles
ax.barh(x, y)                      # Horizontal rectangles
ax.hist(x, bins=10)                # Histogram (Distribution)
ax.boxplot(x)                      # Box and Whisker Plot
ax.axhline(0.45)                   # Horizontal line across axes
ax.axvline(0.65)                   # Vertical line across axes
ax.fill(x, y, color='blue')        # Filled polygons
ax.fill_between(x, y, color='y')   # Fill between y-values
plt.pie([10,20,30], labels=['A','B','C']) # pie chart
```

Plotting Routines (2D Data)

```
ax.imshow(img)                     # Colormapped or RGB array
ax.pcolormesh(data)                # Pseudocolor plot
ax.contour(Y, X, U)                # Contours
ax.contourf(data)                  # Filled contours
ax.pie(x, labels=labels)           # Pie Chart
```

Customization

```
# Colors & Markers
plt.plot(x, y, color='green')      # Color
plt.plot(x, y, linestyle='--')     # Dashed line
plt.plot(x, y, linewidth=4)        # Line width
plt.plot(x, y, marker='o')         # Circle marker
plt.plot(x, y, alpha=0.5)          # Transparency

# Titles & Labels
ax.set(title='An Example Axes',
       ylabel='Y-Axis',
       xlabel='X-Axis')
```

```
# Limits
ax.set_xlim(0, 10.5)
ax.set_ylim(-1.5, 1.5)

# Legends
ax.legend(loc='best')               # Automatic legend positioning

# Ticks
ax.xaxis.set(ticks=range(1,5),
             ticklabels=[3, 100, -12, "foo"])
plt.xticks([0,1,2,3,4,5])            # set x-ticks
plt.yticks([0,2,4,6,8,10])           # set y-ticks
```

Saving & Clearing

```
plt.savefig('foo.png')              # Save figure
plt.savefig('foo.png', transparent=True)
plt.show()                          # Show plot
plt.cla()                           # Clear an axis
plt.clf()                           # Clear the entire figure
plt.close()                         # Close a window
```